TP DE PROGRAMMATION ORIENTEE OBJET :

Exercice 2 : Gestion des étudiants et des résultats académiques

Question 1- Identifions les classes

Réponse :

Le système comporte cinq classes principales : Etudiant, Filiere, UE, Matiere et Note. La classe Etudiant représente l'étudiant inscrit dans une filière et un niveau. La classe Filiere représente la formation suivie. La classe UE représente une unité d'enseignement. La classe Matiere représente une matière d'une UE. La classe Note représente la note obtenue dans une matière.

Question 2 — Définir les relations entre les classes

Réponse : Une Filière regroupe plusieurs Etudiant : c'est une agrégation, car l'étudiant peut exister indépendamment de sa filière. Une Filière est composée d'une ou plusieurs UE : c'est une composition. Une UE est composée d'une ou plusieurs Matiere : c'est également une composition. Un Etudiant possède plusieurs Note. Chaque Note concerne une seule Matiere.

Question 3 — Indiquer les multiplicités UML

Réponse : Une Filière peut regrouper zéro à plusieurs étudiants et chaque étudiant appartient à une seule filière : Filière $1 - 0..*$ Etudiant.

Une filière contient une ou plusieurs UE : Filière $1 - 1..*$ UE.

Une UE contient une ou plusieurs matières : UE $1 - 1..*$ Matiere.

Un étudiant possède zéro à plusieurs notes : Etudiant $1 - 0..*$ Note.

Plusieurs notes peuvent concerner une matière, mais chaque note concerne une seule matière : Note $0..* - 1$ Matiere.

Question 4 — Réaliser le diagramme de classes UML

Réponse : Le diagramme de classes UML est présenté sur le canvas. Il contient les cinq classes, leurs attributs, leurs méthodes, les relations d'association, d'agrégation et de composition, ainsi que les multiplicités.

Diagramme de classes UML : Gestion des étudiants et des résultats académiques

```
@startuml

title Gestion des etudiants et des resultats academiques

skinparam classBorderThickness 2
skinparam ArrowThickness 2
skinparam classAttributeIconSize 0

class Etudiant {
  - matricule : String
  - nom : String
```

EAST-NATI

```
    - niveau : Niveau
    - filiere : Filiere
    - notes : List<Note>
    + calculerMoyenne() : double
    + valideUE(ue : UE) : boolean
    + afficherReleve() : void
  }

  class Filiere {
    - code : String
    - libelle : String
    - ues : List<UE>
    + ajouterUE(ue : UE) : void
  }

  class UE {
    - code : String
    - intitule : String
    - credits : int
    - matieres : List<Matiere>
    + ajouterMatiere(m : Matiere) : void
    + estValidee(etudiant : Etudiant) : boolean
  }

  class Matiere {
    - code : String
    - libelle : String
    - coefficient : double
  }

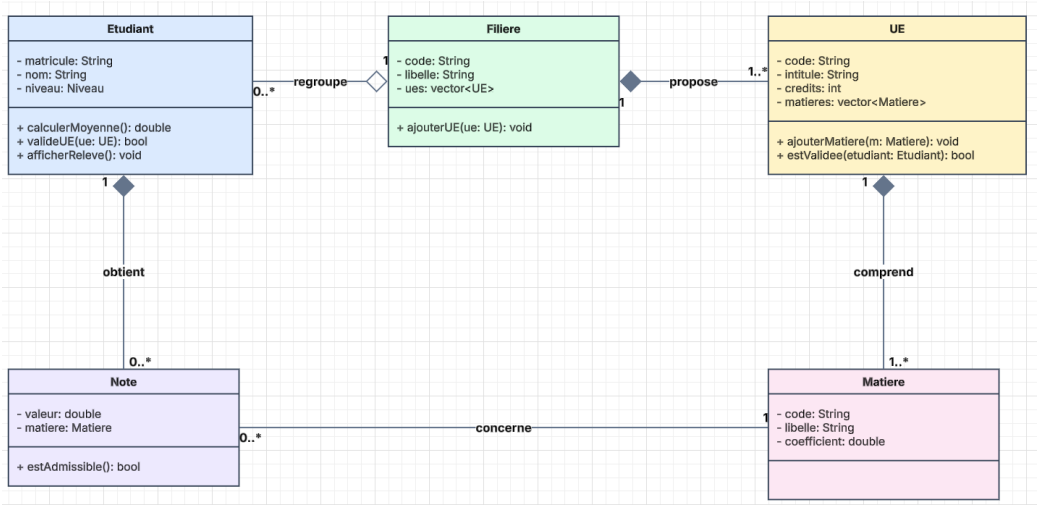
  class Note {
    - valeur : double
    - matiere : Matiere
    + estAdmissible() : boolean
  }

  enum Niveau {
    L1
    L2
    L3
  }

  Filiere "1" o-- "0..*" Etudiant : accueille
  Filiere "1" *-- "1..*" UE : propose
  UE "1" *-- "1..*" Matiere : comprend
  Etudiant "1" *-- "0..*" Note : obtient
  Note "0..*" --> "1" Matiere : concerne
  Etudiant --> Niveau

  @enduml
```

FAST-NATI



Question 5 — Implémenter les classes en C++

Réponse : Les classes sont implémentées dans le bloc de code C++ présent sur le canvas. Les attributs sont privés et les méthodes sont publiques afin de respecter l'encapsulation. Les collections vector permettent de stocker plusieurs UE, matières et notes.

Implémentation des classes en C++

```cpp
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;

enum class Niveau {L1,L2,L3};

class Matiere {
  private:
  string code,libelle;
  double coefficient;
  public:
  Matiere(string c,string l,double coef):code(c),libelle(l),coefficient(coef){}
  const string& getCode()const{return code;}
  const string& getLibelle()const{return libelle;}
  double getCoefficient()const{return coefficient;}
};

class Note {
  private:
  double valeur;
  const Matiere* matiere;
  public:
  Note(double v,const Matiere& m):valeur(v),matiere(&m){}
  double getValeur()const{return valeur;}
  const Matiere& getMatiere()const{return *matiere;}
  bool estAdmissible()const{return valeur>=10.0;}
};

class UE {
  private:
  string code,intitule;
  int credits;
  vector<Matiere> matieres;
  public:
  UE(string c,string i,int cr):code(c),intitule(i),credits(cr){}
  void ajouterMatiere(const Matiere& m){matieres.push_back(m);}
  const string& getCode()const{return code;}
  const string& getIntitule()const{return intitule;}
  const vector<Matiere>& getMatieres()const{return matieres;}

  bool estValidee(const vector<Note>& notes)const{
    double total=0.0,coefficients=0.0;
    for(const Matiere& m:matieres)
    for(const Note& n:notes)
    if(n.getMatiere().getCode()==m.getCode()){
      total+=n.getValeur()*m.getCoefficient();
      coefficients+=m.getCoefficient();
    }
    return coefficients>0.0 && total/coefficients>=10.0;
  }
};

class Filiere {
```

LAST-NAFI

```
56      private:
57      string code,libelle;
58      vector<UE> ues;
59      public:
60      Filiere(string c,string l):code(c),libelle(l){}
61      void ajouterUE(const UE& ue){ues.push_back(ue);}
62      const vector<UE>& getUEs()const{return ues;}
63      const string& getLibelle()const{return libelle;}
64    };
65
66    class Etudiant {
67      private:
68      string matricule,nom;
69      Niveau niveau;
70      const Filiere* filiere;
71      vector<Note> notes;
72      public:
73      Etudiant(string m,string n,Niveau niv,const Filiere& f)
74      :matricule(m),nom(n),niveau(niv),filiere(&f){}
75
76      void ajouterNote(const Note& note){notes.push_back(note);}
77
78      double calculerMoyenne()const{
79        double total=0.0,coefficients=0.0;
80        for(const Note& n:notes){
81          double coef=n.getMatiere().getCoefficient();
82          total+=n.getValeur()*coef;
83          coefficients+=coef;
84        }
85        return coefficients>0.0?total/coefficients:0.0;
86      }
87
88      bool valideUE(const UE& ue)const{return ue.estValidee(notes);}
89
90      void afficherReleve()const{
91        cout<<"Releve de notes - "<<nom<<" ("<<matricule<<")\n";
92        cout<<fixed<<setprecision(2);
93        for(const Note& n:notes)
94        cout<<n.getMatiere().getLibelle()<<" : "<<n.getValeur()<<" / 20\n";
95        cout<<"Moyenne generale : "<<calculerMoyenne()<<" / 20\n";
96      }
97    };
98
99    int main(){
100     Matiere programmation("M101","Programmation C++",2.0);
101     Matiere uml("M102","Modelisation UML",1.0);
102     UE poo("UE01","Programmation et conception",3);
103     poo.ajouterMatiere(programmation);
104     poo.ajouterMatiere(uml);
105     Filiere informatique("INFO","Informatique");
106     informatique.ajouterUE(poo);
107     Etudiant etudiant("E001","Awa Diop",Niveau::L2,informatiqu
108     e);
109     etudiant.ajouterNote(Note(14.0,programmation));
110     etudiant.ajouterNote(Note(12.0,uml));
111     etudiant.afficherReleve();
112     cout<<"UE validee : "<<(etudiant.valideUE(poo)?"oui":"non")<<endl;
113     return 0;
114   }
115
```

Question 6 — Calculer la moyenne d'un étudiant

Réponse : La moyenne est une moyenne pondérée par les coefficients des matières. La formule utilisée est :

$$\text{Moyenne} = \frac{\sum(\text{note} \times \text{coefficient})}{\sum(\text{coefficients})}.$$

Si aucune note n'est enregistrée, la moyenne retournée est 0.

Question 7 — Déterminer si un étudiant valide une UE

Réponse : Le programme sélectionne les notes correspondant aux matières de l'UE, calcule leur moyenne pondérée et compare le résultat au seuil de 10/20. Si la moyenne est supérieure ou égale à 10/20, l'UE est validée.

Question 8 — Afficher le relevé de notes

Réponse : La méthode afficherReleve() affiche le nom et le matricule de l'étudiant, chaque matière avec la note obtenue, la moyenne générale et le résultat de validation de l'UE.

FAST-NATI

Question 9 — Présenter les notions mobilisées

Réponse : L'exercice mobilise l'encapsulation avec les sections private et public, la composition et l'agrégation pour représenter les relations entre objets, les associations entre classes, les tableaux et collections avec vector, les méthodes de calcul pour les moyennes et les multiplicités UML pour préciser le nombre d'objets liés.

Conclusion

Le système obtenu permet de gérer les étudiants, leurs filières, leurs unités d'enseignement, leurs matières et leurs résultats. Le diagramme UML décrit la structure du système et le programme C++ réalise les traitements demandés : calcul de la moyenne, validation d'une UE et affichage du relevé de notes.

TP 2

QUESTION 1 — Identification des acteurs

1.1. Analyse du problème Le système doit centraliser les informations relatives à l'organisation des TD et assurer les fonctions suivantes :

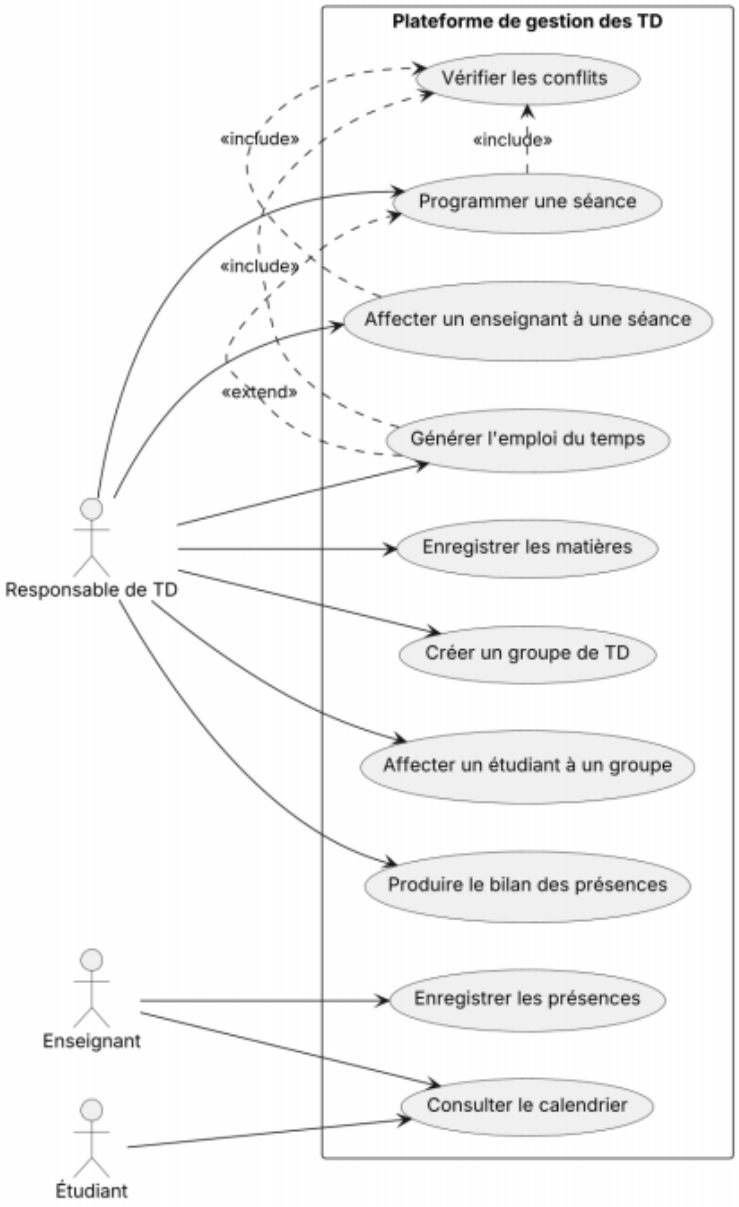
. Enregistrer les matières et gérer les groupes de TD.
. Affecter les étudiants aux groupes.
. Programmer les séances et affecter les enseignants et les salles.
. Enregistrer les présences, absences et retards.
. Consulter le calendrier et produire les bilans de présence.

1.2. Identification des acteurs Administrateur : gère les matières, les étudiants, les enseignants, les groupes, les affectations et les bilans.
Responsable pédagogique : programme les séances, affecte les enseignants, choisit les salles, consulte le calendrier et modifie ou annule une séance.
Enseignant : consulte son emploi du temps et les listes des étudiants, puis enregistre les présences, absences et retards.
Étudiant : consulte son groupe, le calendrier, les horaires, les salles et son historique de présence.

QUESTION 2 — DIAGRAMME DE CAS D'UTILISATION

Acteur	Cas d'utilisation
Administrateur	Gérer les matières, étudiants et enseignants ; créer les groupes ; affecter les étudiants ; produire les bilans.
Responsable pédagogique	Programmer les séances ; affecter les enseignants ; réserver les salles ; consulter le calendrier.
Enseignant	Consulter son emploi du temps ; enregistrer les présences.
Étudiant	Consulter son groupe et le calendrier.

LASTINAT



2.3. Code PlantUML

```
@startuml
left to right direction
actor Administrateur
actor "Responsable pedagogique" as Responsable
actor Enseignant
actor Etudiant

rectangle "Plateforme de gestion des TD" {
  usecase "Gerer les matieres" as UC1
  usecase "Gerer les etudiants" as UC2
  usecase "Gerer les enseignants" as UC3
  usecase "Creer les groupes TD" as UC4
  usecase "Affecter les etudiants" as UC5
  usecase "Programmer une seance" as UC6
  usecase "Affecter un enseignant" as UC7
  usecase "Reserver une salle" as UC8
  usecase "Verifier les disponibilites" as UC9
  usecase "Consulter le calendrier" as UC10
  usecase "Consulter l'emploi du temps" as UC11
  usecase "Enregistrer les presences" as UC12
  usecase "Produire un bilan" as UC13
  usecase "Consulter son groupe" as UC14
}

Administrateur --> UC1
Administrateur --> UC2
Administrateur --> UC3
Administrateur --> UC4
Administrateur --> UC5
Administrateur --> UC13

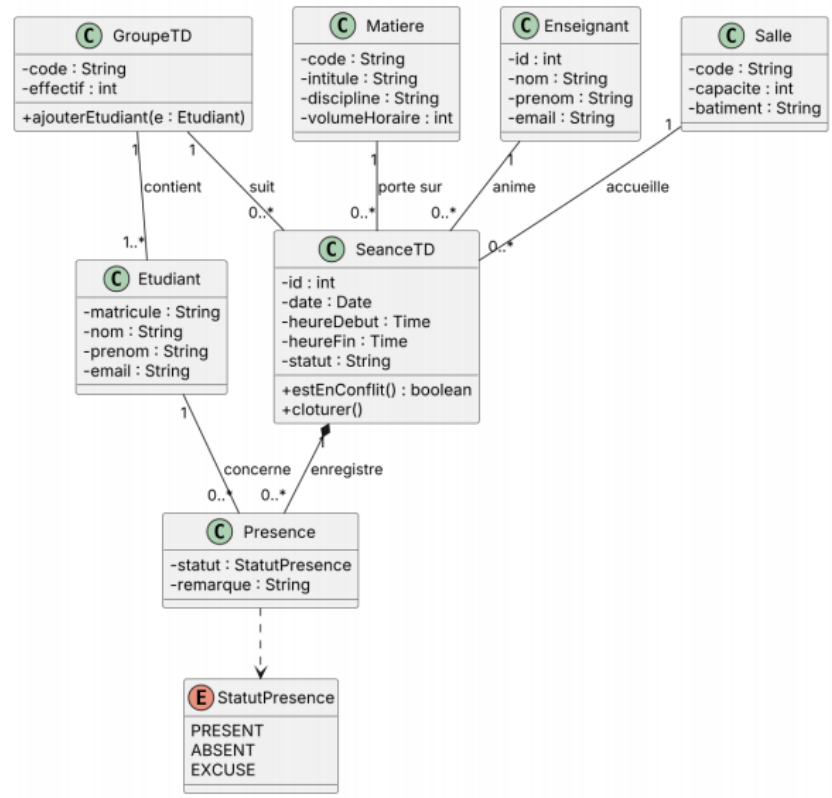
Responsable --> UC6
Responsable --> UC7
Responsable --> UC8
Responsable --> UC10

Enseignant --> UC11
Enseignant --> UC12

Etudiant --> UC10
Etudiant --> UC14

UC6 ..> UC9 : <<include>>
UC7 ..> UC9 : <<include>>
UC8 ..> UC9 : <<include>>
@enduml
```

QUESTION 3 — DIAGRAMME DE CLASSES UML



Remarque : *Presence* est une classe d'association entre *Etudiant* et *SeanceTD* (composition vers la séance : si la séance est supprimée, ses présences le sont aussi).

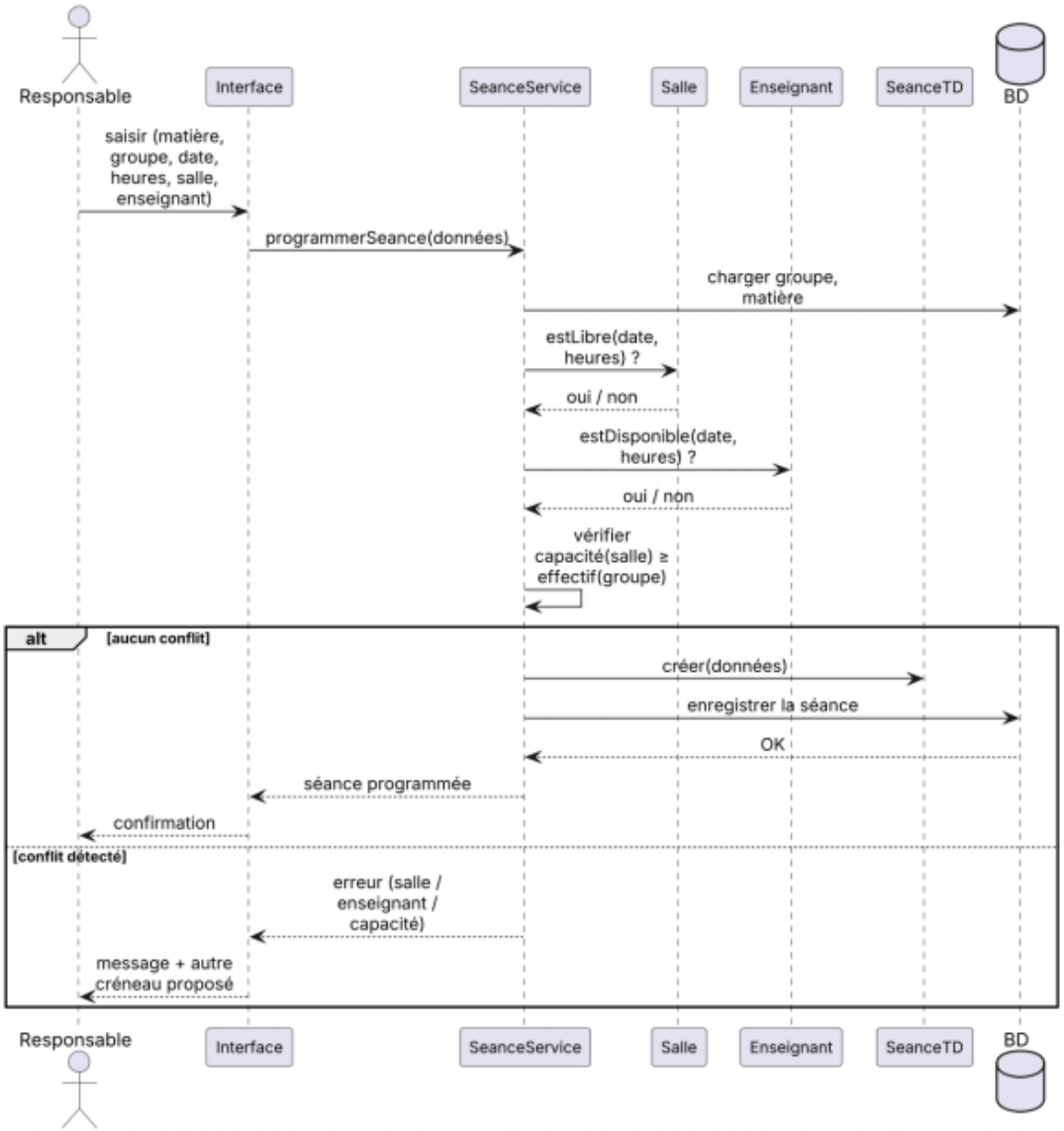
Classe	Attributs	Méthodes
Etudiant	matricule, nom, prenom, email	consulterGroupe(), consulterCalendrier(), consulterPresences()
Matiere	code, libelle, volumeHoraire	ajouterMatiere(), modifierMatiere()
GroupeTD	idGroupe, nomGroupe, niveau, effectifMaximal	ajouterEtudiant(), retirerEtudiant(), afficherListeEtudiants()
SeanceTD	idSeance, date, heureDebut, heureFin, statut	programmerSeance(), modifierSeance(), annulerSeance()
Enseignant	idEnseignant, nom, prenom, specialite, email	consulterPlanning(), enregistrerPresence()
Salle	idSalle, nomSalle, capacite, localisation	verifierDisponibilite(), reserverSalle()
Presence	idPresence, dateEnregistrement, statut	enregistrerPresence(), modifierStatut()

Relations entre les classes

. Un groupe de TD contient plusieurs étudiants ; un étudiant appartient à un groupe à une date donnée.

. Une matière peut être concernée par plusieurs séances.
. Une séance concerne une matière et un groupe.
. Un enseignant peut encadrer plusieurs séances.
. Une séance se déroule dans une seule salle ; une salle peut accueillir plusieurs séances à des horaires différents.
. Une séance possède plusieurs enregistrements de présence ; un étudiant peut avoir plusieurs présences.

QUESTION 4-Diagramme de sequence



. Le responsable se connecte et choisit la programmation d'une séance.
. Il sélectionne la matière, le groupe, l'enseignant et la salle.
. Il indique la date, l'heure de début et l'heure de fin.
. Le système vérifie la disponibilité de la salle et de l'enseignant.
. Si aucune contrainte n'est violée, la séance est enregistrée et une confirmation est envoyée.

. En cas de conflit, le système signale le problème et demande une nouvelle sélection

```
@startuml
actor Responsable
participant "Plateforme TD" as Systeme
participant "Gestion des seances" as Gestion
participant "Gestion des salles" as Salle
participant "Gestion des enseignants" as Enseignant
database "Base de donnees" as BD
Responsable -> Systeme : Se connecter
Systeme --> Responsable : Acces autorise
Responsable -> Systeme : Programmer une seance
Systeme -> Gestion : Demander les informations
Responsable -> Systeme : Saisir date et horaires
Responsable -> Systeme : Choisir groupe et matiere
Responsable -> Systeme : Choisir enseignant et salle
Systeme -> Salle : Verifier disponibilite
Salle -> BD : Consulter reservations
BD --> Salle : Disponibilite de la salle
Salle --> Systeme : Resultat
Systeme -> Enseignant : Verifier disponibilite
Enseignant -> BD : Consulter planning
BD --> Enseignant : Planning enseignant
Enseignant --> Systeme : Resultat
alt Salle et enseignant disponibles
Systeme -> BD : Enregistrer la seance
BD --> Systeme : Confirmation
Systeme --> Responsable : Seance programmee
else Conflit detecte
Systeme --> Responsable : Signaler le conflit
end
@enduml


```

Explication

Le responsable pédagogique déclenche le processus. La plateforme reçoit les informations et demande aux modules concernés de vérifier les disponibilités. La programmation est validée uniquement si l'enseignant et la salle sont libres pendant toute la durée de la séance. Cette vérification évite les doubles réservations.

QUESTION 5 — DIAGRAMME D'ACTIVITÉS

Description du déroulement

. Préparation : le responsable programme la séance et choisit le groupe, la matière, l'enseignant et la salle.

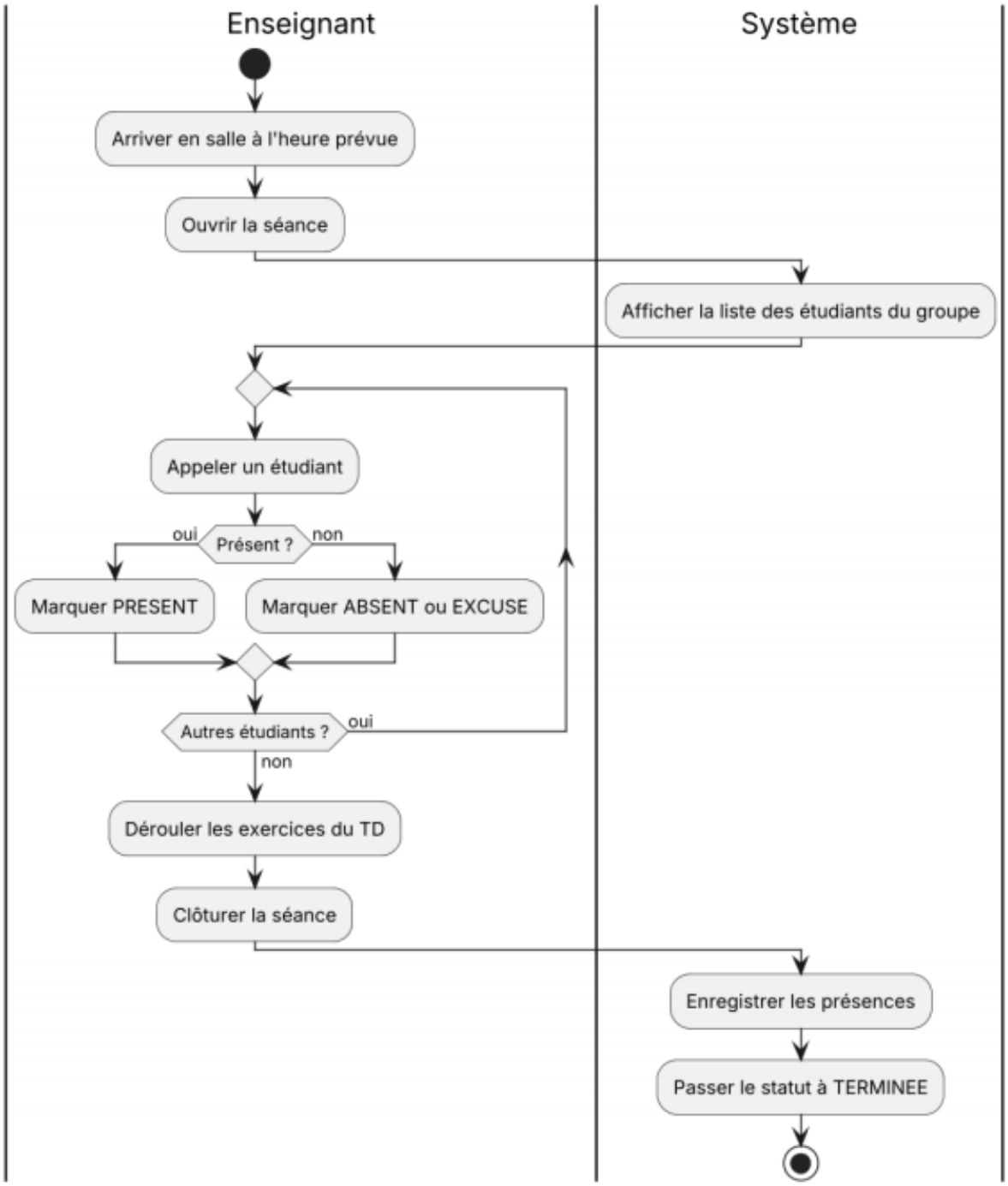
. Vérification : le système contrôle la disponibilité de la salle et de l'enseignant.

. Début : l'enseignant rejoint la salle et commence la séance.

. Présences : l'enseignant enregistre les présences, absences et retards.

. Travail pédagogique : l'enseignant présente les exercices et accompagne les étudiants.

. Clôture : les présences sont sauvegardées et les statistiques sont actualisées.



6. Bilan des présences

Pour un étudiant, sur les séances effectivement tenues de son groupe :

$$\boxed{\textbf{taux de présence} = \text{nombre de PRESENT} \div \text{nombre de séances tenues} \times 100}$$

Vérification : un étudiant présent à 7 séances sur 8 $\rightarrow 7/8 \times 100 =$ **87,5 %** ✓

7. Extension : génération automatique d'un emploi du temps sans conflit

Principe : retour sur trace (*backtracking*). On place les séances une par une ; pour chaque séance, on teste chaque couple (créneau, salle) qui respecte R1 à R4. Si on ne peut plus placer une séance, on revient sur la précédente.

0.1 Pseudocode

```
Fonction possible(i, c, s) : boolen
Si effectif[i] > capacite[s] alors retourner FAUX
Pour j de 0  i-1 faire
Si creneau[j] = c alors
Si salle[j] = s ou enseignant[j] = enseignant[i]
ou groupe[j] = groupe[i] alors
retourner FAUX
FinSi
FinSi
FinPour
retourner VRAI
FinFonction

Fonction placer(i) : boolen
Si i = NB\_SEANCES alors retourner VRAI
Pour c de 0  NB\_CRENEAUX - 1 faire
Pour s de 0  NB\_SALLES - 1 faire
Si possible(i, c, s) alors
creneau[i] <- c ; salle[i] <- s
Si placer(i + 1) alors retourner VRAI
FinSi
FinPour
FinPour
creneau[i] <- -1 ; salle[i] <- -1
retourner FAUX
FinFonction
```

CONCLUSION GÉNÉRALE :
La conception de cette plateforme de gestion des travaux dirigés propose une solution informatique aux difficultés rencontrées dans l'organisation des séances de TD au sein des universités béninoises. Grâce à la modélisation UML, nous avons identifié les acteurs, les fonctionnalités du système, les classes principales et les interactions entre les objets. Le diagramme de cas d'utilisation présente les services proposés aux utilisateurs. Le diagramme de classes décrit la structure statique du système. Le diagramme de séquence explique la programmation d'une séance, tandis que le diagramme d'activités représente le déroulement complet d'un TD. Enfin, la génération automatique de l'emploi du temps tient compte des contraintes de disponibilité. Cette plateforme contribuera à améliorer l'organisation pédagogique, à faciliter le suivi des étudiants, à réduire les conflits de programmation et à rendre la gestion des travaux dirigés plus efficace.